Coinbase Account Policies Part 2

Security Review

Cantina Managed review by:
Giovanni Di Siena, Lead Security Researcher
Red Swan, Security Researcher

March 31, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **4**
3.1 Low Risk . . . 4
3.1.1 Swallowed Policy Hook Reverts Emit No Events . . . 4
3.1.2 Morpho policies overlook non-standard ERC-20 approve semantics . . . 5
3.1.3 `SingleExecutorPolicy::setPolicyManager` ambiguities and executor signature replay . . . 5
3.1.4 `setPolicyManager` Is Insufficient for Mass Migration . . . 5
3.1.5 Policy Authors Have To Implement Manager-Level Routing . . . 6
3.2 Informational . . . 6
3.2.1 ERC-6492 Signature Validation Depends on Solady's Pre-deployed Verifier Contract . 6
3.2.2 Repeated `MorphoLendPolicy` validation can be removed . . . 7
3.2.3 Policies are not checked for persistent code upon installation . . . 7
3.2.4 `PolicyManager` allows the creation of unusable bindings with impossible validity windows . . . 8
3.2.5 `setPolicyManager` Missing Same-Value Check . . . 8

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Mar 16th to Mar 20th the Cantina team conducted a review of account-policies on commit hash af7ff9dd. The team identified a total of **10** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	5	5	0
Gas Optimizations	0	0	0
Informational	5	5	0
Total	**10**	**10**	**0**

2.1 Scope

The security review had the following components in scope for account-policies on commit hash af7ff9dd:

```
src
├── PolicyManager.sol
├── PublicERC6492Validator.sol
└── policies
    ├── MorphoLendPolicy.sol
    ├── MorphoLoanProtectionPolicy.sol
    ├── Policy.sol
    ├── SingleExecutorAuthorizedPolicy.sol
    ├── SingleExecutorPolicy.sol
    └── accounting
        └── RecurringAllowance.sol
```

3 Findings

3.1 Low Risk

3.1.1 Swallowed Policy Hook Reverts Emit No Events

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: When a policy's `onUninstall` or `onReplace` hook reverts and the revert is swallowed (either via the account escape hatch or unconditionally in `_uninstallForReplace`), all four catch blocks silently discard the error. No event is emitted to indicate that the hook reverted, what kind of error occurred, or what the error data was.

The `PolicyUninstalled` event is emitted *after* the catch block in every case, so from an offchain observer's perspective, a clean uninstall and a force-uninstall that swallowed a reverting hook are indistinguishable.

The impact of this is:

1. **No observability for accounts.** An account owner who force-uninstalls a misbehaving policy has no onchain record that the policy's cleanup logic did not run. If the policy was supposed to revoke approvals, release funds, or update external state during `onUninstall`, the account owner has no signal that those side effects were skipped.
2. **No observability for offchain systems.** Monitoring, indexing, and alerting infrastructure cannot distinguish between clean and forced uninstalls. A relayer-submitted uninstall that fails because the policy hook reverted (and then succeeds because the account is the effective caller) looks identical to a cooperative uninstall.
3. **No error classification.** The catch blocks use untyped `catch {}` rather than `catch Error(string memory)`, `catch Panic(uint256)`, or `catch (bytes memory)`. This means even if an event were added, the current structure could not differentiate between a policy that ran out of gas, hit an assertion, reverted with a custom error, or reverted with a reason string. Different error types imply very different root causes (bug vs. intentional access control vs. resource exhaustion) and warrant different responses.

Recommendation: Emit a dedicated event when a policy hook revert is swallowed. Consider using separate catch clauses to capture and emit the error type:

```
event PolicyHookReverted(
    bytes32 indexed policyId,
    address indexed account,
    address indexed policy,
    bytes revertData
);
```

```
// _uninstall Example
try Policy(binding.policy).onUninstall(...) {}
catch (bytes memory revertData) {
    if (effectiveCaller != account) {
        revert Unauthorized(effectiveCaller);
    }
    emit PolicyHookReverted(policyId, account, binding.policy, revertData);
}
```

Using `catch (bytes memory revertData)` captures the raw revert payload for all error types (`Error(string)`, `Panic(uint256)`, and custom errors), making the data available both in the event and for offchain decoding. If finer-grained onchain classification is desired, separate `catch Error(string memory reason)` / `catch Panic(uint256 code)` / `catch (bytes memory lowLevelData)` clauses could each emit distinct events.

Coinbase: Fixed in PR 29 following the recommendation.

Cantina Managed: Fix verified.

3.1.2 Morpho policies overlook non-standard ERC-20 approve semantics

Severity: Low Risk

Context: MorphoLendPolicy.sol#L169, MorphoLoanProtectionPolicy.sol#L228

Description: Certain non-standard ERC-20 tokens, such as USDT, revert when attempting to set a non-zero allowance without first resetting the existing allowance to zero. For such vault assets that require forced approvals, both `MorphoLendPolicy` and `MorphoLoanProtectionPolicy` will generate execution blueprints that will fail if the target account has an outstanding allowance to the given spender.

Recommendation: To avoid the scenario in which policies are unusable until the allowance is manually zeroed, consider adding an additional zero allowance step to the execution blueprint(s).

Coinbase: Fixed by adding additional zero allowance step to call plans in PR 29.

Cantina Managed: Verified. The additional zero allowance step has been added to call plans in both policies.

3.1.3 `SingleExecutorPolicy::setPolicyManager` ambiguities and executor signature replay

Severity: Low Risk

Context: SingleExecutorPolicy.sol#L71-L74

Description: `SingleExecutorPolicy::setPolicyManager` allows the admin to update the policy's `policyManager` pointer to a different version; however, this may be problematic since the old manager retains its own installed/uninstalled records while the policy contract keeps `_configHashByPolicyId`, `_usedNonces`, and any other policy-specific state. This breaks the key invariant that previously uninstalled policies cannot be reinstalled, since the new `PolicyManager` is not necessarily aware of prior context.

Because executor digests are scoped to `(policyId, account, configHash, deadline)` and not to a specific manager, previously unused execution signatures (for which the nonce was not already consumed) can be used on the new manager after the `PolicyManager` configuration change and the same binding is reinstalled.

Uninstall signatures, however, can be replayed by any relayer unconditionally within the deadline even if the binding is not reinstalled on the new manager since it has no such nonce mechanism. Since the `PolicyManager` is updated, uninstall can be made to execute against the new implementation with inconsistent state which is problematic since existing unmigrated policies can be forcibly pre-uninstalled, preventing subsequent reinstallation. Depending on the intended migration behavior, this may violate an invariant or, if the invariant is that uninstalled policies should not be reinstalled then replay will actually enforce this.

This may also cause issues with policy-specific state in which cleanup from one manager can affect the other manager's live installation, e.g. `activePolicyByMarket` and `marketKeyByPolicyId` state in `MorphoLoanProtectionPolicy`.

Recommendation: Clarify the intended behavior and invariants when switching over to a new `PolicyManager`. Also consider including a nonce within the executor uninstall signature typehash to prevent replay of a prior uninstall.

Coinbase: Fixed by removing `setPolicyManager()` entirely in PR 29, so policies can no longer experience more than one immutable `PolicyManager`.

Cantina Managed: Verified. The logic has been removed.

3.1.4 `setPolicyManager` Is Insufficient for Mass Migration

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `setPolicyManager` allows the policy admin to point the policy at a new `PolicyManager`, but it can not migrate the installation state that lives on the old manager. So when the admin calls `setPolicyManager(newManager)`, the policy begins routing execution checks and call plans through the new manager. However, the new manager has no record of any previously installed policyIds. Every user who had the policy installed on the old manager is effectively locked out: their policyId does not exist

on the new manager, so `execute` reverts with `PolicyNotInstalled`. Each user must independently uninstall from the old manager and reinstall on the new one, requiring both a new installation signature (or direct call) and the wallet to add the new manager as an execution-enabled owner.

Recommendation: Document that setPolicyManager is not suitable for live migrations with existing users and is intended only for pre-user operational changes (e.g., pointing a freshly deployed policy at the correct manager). If mass migration is a future requirement, the protocol would benefit from an architecture where installation state is not coupled to a specific PolicyManager deployment.

Coinbase: Fixed by removing `setPolicyManager` entirely in PR 29. Upon deeper exploration of a potential migration, we believe mutable policy managers will always pose more complexity than other possibilities such as deployments of a separate, parallel system.

Cantina Managed: Verified. We agree with this assessment.

3.1.5 Policy Authors Have To Implement Manager-Level Routing

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `installWithSignature` and `replaceWithSignature` unconditionally invoke `_execute` regardless of whether the caller supplied non-empty `executionData`. Because the manager does not distinguish between an install-only intent and an install-and-execute intent, that routing decision is silently delegated to the policy itself. Every conforming policy must therefore implement a check:

```
function _onExecute(..., bytes calldata executionData, ...)
    internal override returns (bytes memory accountCallData, bytes memory postCallData)
{
    if (executionData.length == 0) return (accountCallData, postCallData); // required
    ↪  check
    // ...
}
```

This check is an undocumented protocol-level convention that every policy author must independently discover, understand, and correctly implement.

The validity window check compounds the problem. It is deferred (running only when `accountCallData` is non-empty) specifically so that install-only flows before `validAfter` do not revert. This means the manager cannot enforce its own invariants: it cannot distinguish "the policy chose not to act" from "this was an install-only call that should never have reached the policy.".

Recommendation: Consider checking the execution data length and only dispatching `_execute` if the data is non-empty in `installWithSignature` and `replaceWithSignature`. Furthermore, consider moving the validity check in `_execute` to before when `onExecute` is called on the policy.

After these changes, every invocation of `onExecute` represents a genuine, manager-validated execution request. A revert from `onExecute` is always meaningful. A policy that does not handle some `executionData` format produces a loud `abi.decode` revert rather than a silent state corruption or no-op.

Coinbase: Fixed in PR 29.

Cantina Managed: Fix verified.

3.2 Informational

3.2.1 ERC-6492 Signature Validation Depends on Solady's Pre-deployed Verifier Contract

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `PublicERC6492Validator.isValidSignatureNowAllowSideEffects` is a wrapper around Solady's `SignatureCheckerLib.isValidERC6492SignatureNowAllowSideEffects` function, which internally hardcodes a call to a "non-reverting verifier" at address `0x0000bc370E4DC924F427d84e2f4B9Ec81626ba7E`:

```
call(
    gas(),
    0x0000bc370E4DC924F427d84e2f4B9Ec81626ba7E, // Non-reverting verifier.
    0,
    m,
    add(returndatasize(), 0x40),
    staticcall(gas(), 4, add(signature, 0x20), n, add(m, 0x40), n),
    0x00
)
isValid := returndatasize()
```

This verifier contract is required for ERC-6492 signature validation of counterfactual (not-yet-deployed) smart contract accounts. It must therefore be independently deployed on every chain where a PolicyManager is deployed. If the verifier is absent the attempt to validate a signature will produce an `InvalidSignature` error regardless of what signature it attempts to validate.

Recommendation: Document the Solady verifier deployment as a prerequisite in the deployment runbook. Documentation on how to deploy it yourselves can be found in Vectorized's 011d6bec gist if it does not exist. Consider adding a constructor or deployment-time check that verifies code exists at `0x0000bc370E4DC924F427d84e2f4B9Ec81626ba7E`, similar to the existing check on the `PublicERC6492Validator` address in the `PolicyManager` constructor.

Coinbase: Fixed in PR 29.

Cantina Managed: Verified. The constructor now reverts if the Solady verifier is not deployed on the given chain.

3.2.2 Repeated `MorphoLendPolicy` validation can be removed

Severity: Informational

Context: RecurringAllowance.sol#L83, MorphoLendPolicy.sol#L151-L159

Description: `MorphoLendPolicy` reverts if the decoded policy `actionData` contains a zero deposit amount; however, this is not necessary as the validation is already performed within the `RecurringAllowance` library when `useLimit()` is called.

Recommendation: Remove the repeated validation from the policy.

Coinbase: Fixed in PR 29.

Cantina Managed: Verified. The repeated validation has been removed.

3.2.3 Policies are not checked for persistent code upon installation

Severity: Informational

Context: PolicyManager.sol#L577

Description: Unlike other `PolicyManager` and `Policy` instances that call `_isNotPersistentCode()`, policies themselves are never validated to have persistent code upon installation. For high-level external calls to addresses with no code, Solidity inserts a codesize check that will revert. If a policy address has revoked an EIP-7702 delegation, this means:

- The `onUninstall()` hook call will revert, but the revert is swallowed and forced uninstall still succeeds assuming the effective caller is the account.
- The `_uninstallForReplace()` function swallows hook reverts unconditionally, so replacement can still proceed skipping policy cleanup.
- The `onExecute()` hook is not called within a `try/catch`, so execution will be blocked until the account force uninstalls the problematic policy.

Recommendation: Consider checking newly-installed policy addresses for persistence.

Coinbase: Fixed in PR 29.

Cantina Managed: Verified. Policies are now validated to have persistent code upon installation.

3.2.4 `PolicyManager` allows the creation of unusable bindings with impossible validity windows

Severity: Informational

Context: PolicyManager.sol#L564-L567

Description: The `PolicyManager` performs validation on the supplied `validUntil` to reject already-expired bindings; however, it fails to enforce `validAfter <= validUntil` which allows for the creation of permanently unusable bindings due to executions reverting in `_checkValidityWindow()`.

This impossible validity window may also be used by the policies directly, as in the case of `MorphoLendPolicy`, feeding invalid bounds into recurring allowance logic. In this case, if execution were possible, the policy would revert in any case but it would be better to prevent the creation of such bindings to begin with.

Recommendation: Consider explicitly reverting when an the supplied validity window is invalid.

Coinbase: Fixed in PR 29.

Cantina Managed: Verified. Invalid validity windows are now rejected upon installation and installations during replacement.

3.2.5 `setPolicyManager` Missing Same-Value Check

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `setPolicyManager` does not check whether `newPolicyManager` is the same as the current `policyManager` before writing the new value to storage and emitting `PolicyManagerUpdated`. Off-chain monitoring systems that track `PolicyManagerUpdated` events would interpret the emission as a genuine manager change, which could trigger unnecessary alerts or state updates.

Recommendation: Add a same-value check before the assignment.

Coinbase: Fixed by way of removing `setPolicyManager()` entirely in PR 29.

Cantina Managed: Verified.